volume, which effectively becomes a local gem cache. Remember that a mounted volume overlays, and is separate from, the container's filesystem; its files persist beyond the life cycle of the container itself.

Let's see how this works in practice.

First, we need to configure Bundler to use an explicit, known directory for installing gems to…/gems, say. We do this by setting the BUNDLE_PATH environment variable. Let's update our Dockerfile as follows:

```
COPY Gemfile* /usr/src/app/
WORKDIR /usr/src/app
ENV BUNDLE_PATH /gems
RUN bundle install
COPY . /usr/src/app/
CMD ["bin/rails", "s", "-b", "0.0.0.0"]
```

The real magic, though, comes in our docker-compose.yml file:

```
version: '3'

services:
  web:
    build: .
    ports:
      - "3000:3000"
      - "4000:4000"
    volumes:
      - .:/usr/src/app
      - gem_cache:/gems
    env_file:
      - .env/development/web
      - .env/development/database
    environment:
      - WEBPACKER_DEV_SERVER_HOST=webpack_dev_server

  webpack_dev_server:
    build: .
    command: ./bin/webpack-dev-server
    ports:
      - 3035:3035
    volumes:
      - .:/usr/src/app
      - gem_cache:/gems
    env_file:
      - .env/development/web
      - .env/development/database
    environment:
      - WEBPACKER_DEV_SERVER_HOST=0.0.0.0
```